

AI Homework Supervision Bot

Software Build Plan

Complete Technical Specification for Implementation

Purpose of this document: This is a precise, self-contained specification for building the software system. Every module, class, method, file path, and dependency is defined here. Claude Code should be able to implement the full system from this document alone.

Scope	Details
Target platform	PC / Laptop / Tablet with webcam (no hardware bot yet)
Primary language	Python 3.11+
OS	Windows 10/11, macOS 13+, or Ubuntu 22.04
Goal	Working end-to-end system: webcam → detection → voice alerts → parent dashboard
Excluded (Phase 2+)	Hardware bot, WeChat Mini Program, cloud sync, LLM script gen

June 2026

1. Project Directory Structure

Create this exact folder structure. Every module maps to a file listed here.

```

supervision_bot/
  main.py          # Entry point – starts full pipeline
  config.py        # All configurable constants (thresholds, durations)
  requirements.txt  # Python dependencies
  vision/
    __init__.py
    capture.py      # Module 1: Camera capture loop
    mediapipe_module.py # Module 2a: Face Mesh + Pose via MediaPipe
    yolo_module.py  # Module 2b: YOLOv8n object detection
    optical_flow.py # Module 2c: Writing activity via optical flow
    feature_aggregator.py # Module 3: Combines all signals → FeatureVector
  logic/
    __init__.py
    state_machine.py # Module 4: Pomodoro state machine
    rules_engine.py  # Module 5: Event detection from FeatureVector
    session_manager.py # Coordinates state machine + rules engine
  output/
    __init__.py
    script_bank.py   # Module 6a: Template scripts (Chinese + English)
    tts.py           # Module 6b: Text-to-speech (edge-tts + fallback)
    expression.py    # Module 6c: Console/GUI expression feedback
  data/
    __init__.py
    db.py            # Module 7: SQLite schema + CRUD operations
    supervision.db   # Auto-created on first run (gitignore this)
  api/
    __init__.py
    server.py        # Module 8: FastAPI server
    schemas.py       # Pydantic models for API
  dashboard/         # Module 9: React frontend (separate npm project)
    package.json
    src/
      App.jsx
      components/
        StatusCard.jsx
        EventTimeline.jsx
        FocusChart.jsx
        SessionSummary.jsx
      api/
        client.js    # API calls to FastAPI backend

```

2. Dependencies & Installation

2.1 requirements.txt (Python)

```
# Core CV
opencv-python==4.9.0.80
mediapipe==0.10.14
ultralytics==8.2.0

# State machine
transitions==0.9.0

# TTS
edge-tts==6.1.10
pyttsx3==2.90
pygame==2.5.2

# API / backend
fastapi==0.111.0
uvicorn[standard]==0.30.1
pydantic==2.7.1
python-multipart==0.0.9

# Data
aiosqlite==0.20.0
sqlalchemy==2.0.30

# Utilities
numpy==1.26.4
Pillow==10.3.0
asyncio-mqtt==0.16.2
```

2.2 Install Commands

```
# Create virtual environment
python -m venv venv
source venv/bin/activate      # Linux/Mac
venv\Scripts\activate        # Windows

# Install Python deps
pip install -r requirements.txt

# Install YOLOv8 model (downloads automatically on first run)
# No manual download needed – ultralytics fetches yolov8n.pt on first use

# Frontend (run inside dashboard/ folder)
cd dashboard
npm install
```

2.3 Verify Installation

```
python -c "import cv2, mediapipe, ultralytics, transitions, edge_tts, fastapi; print('All OK!')"
# Expected output: All OK
```



```
API_PORT            = 8000
DATABASE_URL        = "sqlite+aiosqlite:///data/supervision.db"

# ■■ Child Profile (default – override via UI or CLI) ████████████████████████████████████████
CHILD_NAME          = "■■"      # Default if no name set
CHILD_AGE           = 10
```

4. Vision Modules (vision/)

4.1 capture.py — Camera Capture Loop

Provides a thread-safe frame queue. All other vision modules read from this queue. Never block the main thread on camera I/O.

```
class CameraCapture:
    """Runs in a background thread. Puts frames into a queue."""

    def __init__(self, camera_index=CAMERA_INDEX, width=FRAME_WIDTH, height=FRAME_HEIGHT, fps=TARGET_FPS):
    def start(self) -> None          # Start background thread
    def stop(self) -> None           # Signal thread to stop, release camera
    def get_frame(self) -> np.ndarray | None  # Return latest frame or None
    def is_running(self) -> bool

    # Internal
    def _capture_loop(self) -> None  # Background thread: cap.read() -> queue
```

■ Use `threading.Thread` with `daemon=True`. Use a `collections.deque(maxlen=2)` as the buffer — always keep only the latest 2 frames to avoid memory buildup.

4.2 mediapipe_module.py — Face Mesh + Pose

Wraps MediaPipe solutions. Returns structured results, not raw landmarks.

```
from dataclasses import dataclass

@dataclass
class FaceResult:
    detected: bool
    confidence: float
    head_pitch: float      # degrees, + = nodding down, - = tilting up
    head_yaw: float        # degrees, + = turning right, - = turning left
    ear_left: float        # Eye Aspect Ratio, left eye (0.0 to 0.5)
    ear_right: float       # Eye Aspect Ratio, right eye
    ear_avg: float         # Average of both
    gaze_x: float          # Normalized iris x offset (-1 left to +1 right)
    gaze_y: float          # Normalized iris y offset

@dataclass
class PoseResult:
    detected: bool
    confidence: float
    seat_present: bool     # Are shoulders visible and in lower 2/3 of frame?
    slouch_score: float    # 0.0 = upright, 1.0 = fully slouched
    distance_estimate: float # Rough face-to-camera distance estimate (arbitrary units)

class MediaPipeModule:
    def __init__(self):
    def process(self, frame: np.ndarray) -> tuple[FaceResult, PoseResult]
    def draw_debug(self, frame: np.ndarray, face: FaceResult, pose: PoseResult) -> np.ndarray

    # Internal helpers
    def _compute_ear(self, landmarks, eye_indices: list[int]) -> float
    def _compute_head_pose(self, landmarks, frame_shape) -> tuple[float, float]
    def _compute_gaze(self, landmarks) -> tuple[float, float]
    def _compute_seat_presence(self, pose_landmarks, frame_shape) -> bool
```

■ For head pose: use `cv2.solvePnP` with 6 reference 3D points (nose tip=index 1, chin=152, left eye corner=226, right eye corner=446, left mouth=57, right mouth=287) and the standard 3D face model array.

4.3 yolo_module.py — Object Detection

```
from dataclasses import dataclass

@dataclass
class YoloResult:
    phone_detected: bool
    phone_confidence: float
    person_detected: bool
    book_detected: bool
    raw_detections: list[dict]  # [{"class": str, "conf": float, "bbox": [x1,y1,x2,y2]}]

class YoloModule:
    def __init__(self, model_path: str = "yolov8n.pt")
    def process(self, frame: np.ndarray) -> YoloResult
    def draw_debug(self, frame: np.ndarray, result: YoloResult) -> np.ndarray

    # Internal
    def _load_model(self) -> None  # Loads ultralytics YOLO model on first call
```

■ Run YOLO at 5fps (every 3rd frame at 15fps) to reduce CPU load. Cache the last result for frames where YOLO doesn't run. Phone detection does not need 15fps — once every 200ms is sufficient.

4.4 optical_flow.py — Writing Activity

```
class OpticalFlowModule:
    def __init__(self, roi: tuple[int,int,int,int] | None = None)
    def set_roi(self, x1: int, y1: int, x2: int, y2: int) -> None
    def process(self, frame: np.ndarray) -> float
        """Returns writing_activity_score (0.0 to 100.0).
        Higher = more motion in the desk ROI.
        If ROI not set, uses full lower-center of frame as default."""
    def reset(self) -> None  # Call at start of each focus session

    # Internal
    def _compute_farneback(self, prev_gray, curr_gray) -> float
```

4.5 feature_aggregator.py — Combined Feature Vector

```
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class FeatureVector:
    timestamp: datetime
    # From MediaPipe
    face_present: bool = False
    face_confidence: float = 0.0
    head_pitch: float = 0.0
    head_yaw: float = 0.0
    ear_avg: float = 0.3
    gaze_x: float = 0.0
    seat_present: bool = True
    slouch_score: float = 0.0
    distance_estimate: float = 50.0
    # From YOLO
    phone_detected: bool = False
```

```
    phone_confidence: float = 0.0
    # From optical flow
    writing_score: float = 50.0
    # Derived
    is_distracted: bool = False    # Convenience flag (computed by aggregator)
    is_fatigued: bool = False

class FeatureAggregator:
    def __init__(self, window_seconds: int = 5)
    def update(self, face: FaceResult, pose: PoseResult, yolo: YoloResult,
               flow_score: float) -> FeatureVector
        """Combines all module outputs. Applies rolling average over window_seconds.
        Sets is_distracted = True if: face not present for >2s OR
        abs(gaze_x) > 0.4 sustained for >3s."""
    def get_latest(self) -> FeatureVector | None
    def reset(self) -> None
```

5. Logic Modules (logic/)

5.1 state_machine.py — Pomodoro Flow

Use the **transitions** library. Install: `pip install transitions`

```
from transitions import Machine

STATES = ["IDLE", "PREPARE", "FOCUS", "BREAK", "RESUME_CHECK", "COMPLETE"]

TRANSITIONS = [
    {"trigger": "start_session", "source": "IDLE", "dest": "PREPARE"},
    {"trigger": "ready", "source": "PREPARE", "dest": "FOCUS"},
    {"trigger": "focus_complete", "source": "FOCUS", "dest": "BREAK"},
    {"trigger": "break_complete", "source": "BREAK", "dest": "RESUME_CHECK"},
    {"trigger": "child_returned", "source": "RESUME_CHECK", "dest": "FOCUS"},
    {"trigger": "session_end", "source": "*", "dest": "COMPLETE"},
    {"trigger": "reset", "source": "*", "dest": "IDLE"},
]

class PomodoroMachine:
    def __init__(self, on_state_change: Callable[[str, str], None] | None = None)
        """on_state_change(old_state, new_state) callback fired on every transition."""

    # Public API called by session_manager.py
    def tick(self) -> None
        """Call every second. Checks timer and auto-fires transitions when timer expires."""
    def force_break(self) -> None
        """Called by rules engine if 3+ severe violations in focus period."""
    def manual_start(self) -> None
        """Called when user presses Start button."""

    # State queries
    @property
    def current_state(self) -> str
    @property
    def elapsed_seconds(self) -> int # Seconds in current state
    @property
    def round_number(self) -> int # Current Pomodoro round (starts at 1)
    @property
    def total_focus_today(self) -> int # Total focused seconds this session
```

■ *on_state_change* callback is how the *session_manager* triggers TTS announcements like 'Great work! Take a 5-minute break.' on entering *BREAK* state.

5.2 rules_engine.py — Event Detection

Reads `FeatureVector` each tick. Applies thresholds. Emits Event objects.

```
from dataclasses import dataclass
from enum import Enum

class EventType(Enum):
    PHONE_DETECTED = "phone_detected"
    OUT_OF_SEAT = "out_of_seat"
    DISTRACTED_GAZE = "distracted_gaze"
    WRITING_IDLE = "writing_idle"
    FATIGUE_DROWSY = "fatigue_drowsy"
    BAD_POSTURE_CLOSE = "bad_posture_close"
    BAD_POSTURE_SLOUCH = "bad_posture_slouch"
    BREAK_STUDYING = "break_studying"
```

```

BREAK_SEDENTARY = "break_sedentary"
GOOD_FOCUS      = "good_focus"          # Positive event (no alert for X minutes)

class Priority(Enum):
    LOW      = 1    # Expression change only, no voice
    MEDIUM  = 2    # Gentle voice reminder
    HIGH     = 3    # Firm voice + log for parent

@dataclass
class Event:
    event_type: EventType
    priority: Priority
    message_intent: str      # e.g. "phone_detected_high" — key into script_bank
    context: dict            # Extra data: {"duration_seconds": 12, ...}
    timestamp: datetime

class RulesEngine:
    def __init__(self):

    def evaluate(self, fv: FeatureVector, pomodoro_state: str) -> list[Event]:
        """
        Main method. Call every second during FOCUS and BREAK states.
        Returns list of Events to fire (empty list = no alerts).
        Internally tracks sustained durations for each signal.
        Applies cooldown: same EventType cannot fire again within ALERT_COOLDOWN_SEC.
        Applies cap: if total events fired this session >= MAX_ALERTS_PER_SESSION,
                     only HIGH priority events pass through.
        """

    def reset_session(self) -> None    # Call at start of each new session
    def get_session_events(self) -> list[Event] # All events fired this session

    # Internal rule methods (each returns Event | None)
    def _check_phone(self, fv: FeatureVector) -> Event | None
    def _check_out_of_seat(self, fv: FeatureVector) -> Event | None
    def _check_distraction(self, fv: FeatureVector) -> Event | None
    def _check_writing_idle(self, fv: FeatureVector) -> Event | None
    def _check_fatigue(self, fv: FeatureVector) -> Event | None
    def _check_posture(self, fv: FeatureVector) -> Event | None
    def _check_good_focus(self, fv: FeatureVector) -> Event | None
    def _check_break_rules(self, fv: FeatureVector) -> list[Event]

```

5.3 session_manager.py — Coordinator

Ties everything together. This is the main loop that runs during a session.

```

class SessionManager:
    """
    Owns all modules. Runs the main async loop.
    Called from main.py.
    """

    def __init__(self):
        self.capture      = CameraCapture()
        self.mp_module    = MediaPipeModule()
        self.yolo_module  = YoloModule()
        self.flow_module  = OpticalFlowModule()
        self.aggregator   = FeatureAggregator()
        self.pomodoro     = PomodoroMachine(on_state_change=self._on_state_change)
        self.rules        = RulesEngine()

```

```
self.tts          = TTSEngine()
self.scripts      = ScriptBank()
self.db           = Database()

async def run(self) -> None
    """Main async loop. Runs until session ends."""

def _process_frame(self, frame: np.ndarray) -> None
    """Per-frame processing: CV pipeline → feature aggregation."""

def _tick_logic(self) -> None
    """Per-second: rules evaluation → TTS → DB logging."""

def _on_state_change(self, old: str, new: str) -> None
    """Fires TTS announcement when Pomodoro state changes."""

def _handle_event(self, event: Event) -> None
    """Look up script for event, fire TTS, log to DB."""

def start_session(self, child_name: str) -> None
def end_session(self) -> dict    # Returns session summary dict
```


6.2 tts.py — Text-to-Speech Engine

```
class TTSEngine:
    """
    Primary: edge-tts (requires internet). Async.
    Fallback: pyttsx3 (offline, synchronous).
    Cache: pre-generated MP3s for all template scripts stored in TTS_CACHE_DIR.
    """

    def __init__(self):
        self.cache_dir = Path(TTS_CACHE_DIR)
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        self._engine_fallback = pyttsx3.init() # Initialized once

    async def speak(self, text: str) -> None
        """
        1. Check cache for text hash – if cached MP3 exists, play it.
        2. Try edge-tts: await edge_tts.Communicate(text, TTS_VOICE).save(path)
        3. On failure: fallback to pyttsx3.
        Play audio via pygame.mixer (non-blocking).
        """

    async def pre_cache_all(self, script_bank: ScriptBank) -> None
        """Pre-generate MP3 for every script in script_bank at startup.
        Runs as background task so it does not block session start."""

    def stop(self) -> None # Stop currently playing audio

    # Internal
    def _cache_key(self, text: str) -> str # MD5 hash of text → filename
    async def _play_mp3(self, path: Path) -> None
```

6.3 expression.py — Visual Feedback

For the software version (no TFT hardware), display a simple OpenCV overlay window showing the bot's current expression state as text + emoji. This placeholder will be replaced by actual TFT rendering in Phase 3.

```
EXPRESSION_MAP = {
    "IDLE":      ("■", "Standby",      (100, 100, 100)),
    "PREPARE":   ("■", "Getting Ready", (50, 150, 50)),
    "FOCUS":     ("■", "Focused",      (50, 100, 200)),
    "CONCERNED": ("■", "Hmm...",      (200, 150, 50)),
    "PHONE_ALERT": ("■", "No Phone!",  (200, 50, 50)),
    "GOOD_FOCUS": ("■", "Great Work!", (50, 180, 50)),
    "BREAK":     ("■", "Break Time",   (50, 180, 100)),
    "FATIGUE":   ("■", "Stay Awake!", (180, 100, 50)),
}

class ExpressionDisplay:
    def __init__(self, window_name: str = "Bot Expression")
    def show(self, state: str) -> None
        """Renders expression overlay in OpenCV window (non-blocking)."""
    def update_debug_frame(self, frame: np.ndarray) -> None
        """Optional: overlay debug info (feature vector values) on camera frame."""
    def close(self) -> None
```

7. Data Module (data/db.py)

SQLite via SQLAlchemy async. Schema is created automatically on first run.

7.1 Schema

```
# SQLAlchemy ORM models

class Child(Base):
    __tablename__ = "children"
    id            : int (PK, autoincrement)
    name          : str (not null)
    age           : int
    created_at    : datetime

class Session(Base):
    __tablename__ = "sessions"
    id             : int (PK)
    child_id       : int (FK → children.id)
    started_at     : datetime
    ended_at       : datetime | None
    rounds_completed : int (default 0)
    total_focus_seconds : int (default 0)
    focus_score_avg : float | None # 0-100

class Event(Base):
    __tablename__ = "events"
    id             : int (PK)
    session_id     : int (FK → sessions.id)
    event_type      : str # EventType.value
    priority        : str # Priority.value
    timestamp       : datetime
    duration_seconds : int | None

class FocusSnapshot(Base):
    __tablename__ = "focus_snapshots"
    id             : int (PK)
    session_id     : int (FK → sessions.id)
    minute_mark    : int # Minutes elapsed in session
    writing_score    : float
    face_present    : bool
    phone_detected  : bool
```

7.2 Database Class

```
class Database:
    def __init__(self, url: str = DATABASE_URL)
    async def init(self) -> None # Create tables if not exist
    async def create_child(self, name: str, age: int) -> int # Returns child_id
    async def start_session(self, child_id: int) -> int # Returns session_id
    async def end_session(self, session_id: int, summary: dict) -> None
    async def log_event(self, session_id: int, event: Event) -> None
    async def log_snapshot(self, session_id: int, minute: int, fv: FeatureVector) -> None
    async def get_session(self, session_id: int) -> dict
    async def get_events(self, session_id: int) -> list[dict]
    async def get_history(self, child_id: int, days: int = 7) -> list[dict]
    async def get_current_session(self, child_id: int) -> dict | None
```

8. FastAPI Backend (api/)

Runs on localhost:8000. The React dashboard calls these endpoints. Also used for session control (start/stop) from the dashboard.

Method	Endpoint	Description	Response
GET	/session/current	Get current active session status	SessionStatus schema
POST	/session/start	Start a new Pomodoro session	{ session_id, child_name }
POST	/session/stop	End the current session	SessionSummary schema
GET	/session/{id}/events	Get all events for a session	List[EventOut]
GET	/session/{id}/snapshots	Get per-minute focus data	List[SnapshotOut]
GET	/history/{child_id}	7-day history for charts	List[SessionOut]
GET	/children	List all child profiles	List[ChildOut]
POST	/children	Create a child profile	ChildOut
GET	/stream/features	SSE stream of live FeatureVector	text/event-stream

8.1 Key Schemas (schemas.py)

```
class SessionStatus(BaseModel):
    session_id: int | None
    pomodoro_state: str          # IDLE / PREPARE / FOCUS / BREAK / etc.
    elapsed_seconds: int
    round_number: int
    child_name: str
    is_focused: bool
    last_event: str | None
    focus_score_current: float    # Rolling 5-min score 0-100

class EventOut(BaseModel):
    event_type: str
    priority: str
    timestamp: datetime
    duration_seconds: int | None

class SessionSummary(BaseModel):
    session_id: int
    total_focus_seconds: int
    rounds_completed: int
    focus_score_avg: float
    event_counts: dict[str, int]  # {"phone_detected": 2, "distracted_gaze": 5, ...}
    ai_summary: str               # Template-generated text summary
```

8.2 SSE Live Feed

The /stream/features endpoint pushes a FeatureVector JSON every second via Server-Sent Events. The React dashboard listens to this for the live status indicator without polling.

```
# In server.py
from sse_starlette.sse import EventSourceResponse

@app.get("/stream/features")
async def stream_features(request: Request):
    async def generator():
        while True:
```

```
        if await request.is_disconnected():
            break
        fv = session_manager.aggregator.get_latest()
        if fv:
            yield {"data": fv.model_dump_json()}
            await asyncio.sleep(1)
    return EventSourceResponse(generator())
```

9. React Dashboard (dashboard/)

Vite + React + TailwindCSS. Single page app. Runs on port 5173 (dev) or served statically by FastAPI in production.

9.1 package.json Dependencies

```
{
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "recharts": "^2.12.7",
    "lucide-react": "^0.383.0",
    "tailwindcss": "^3.4.4"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.3.1"
  }
}
```

9.2 Component Breakdown

Component	File	What It Shows	Data Source
StatusCard	StatusCard.jsx	Current state badge, elapsed time, round number, child name	GET /session/current (poll 5s) + SSE /stream/features
FocusIndicator	StatusCard.jsx	Live dot: green=focused, orange=distracted, red=phone/absent	SSE /stream/features
EventTimeline	EventTimeline.jsx	Scrollable list of today's events with time, type, icon	GET /session/{id}/events
FocusChart	FocusChart.jsx	Line chart: focus score per minute this session	GET /session/{id}/snapshots — use Recharts LineChart
WeekChart	FocusChart.jsx	Bar chart: daily focus minutes past 7 days	GET /history/{child_id} — use Recharts BarChart
SessionSummary	SessionSummary.jsx	End-of-session card: rounds, total time, event counts, text summary	GET /session/current after state=COMPLETE
StartButton	App.jsx	Large start/stop button. POST /session/start or /session/stop	POST /session/start stop

9.3 CORS Configuration

Add this to FastAPI server.py so the React dev server can call the API:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

10. main.py — Entry Point

```
"""
main.py — Start the AI Homework Supervision Bot

Usage:
python main.py                # Start with default child profile
python main.py --name ■■ --age 10
python main.py --debug        # Show camera feed with debug overlay
python main.py --api-only     # Start API server without CV pipeline (for testing)
"""

import asyncio
import argparse
import uvicorn
from logic.session_manager import SessionManager
from api.server import app, set_session_manager
from data.db import Database
import config

async def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--name",      default=config.CHILD_NAME)
    parser.add_argument("--age",       type=int, default=config.CHILD_AGE)
    parser.add_argument("--debug",     action="store_true")
    parser.add_argument("--api-only",  action="store_true")
    args = parser.parse_args()

    db = Database()
    await db.init()

    session_manager = SessionManager(debug=args.debug)
    set_session_manager(session_manager)  # Inject into FastAPI app

    # Start FastAPI in background
    api_task = asyncio.create_task(
        uvicorn.serve(app, host=config.API_HOST, port=config.API_PORT)
    )

    if not args.api_only:
        await session_manager.run()  # Blocks until session ends

    await api_task

if __name__ == "__main__":
    asyncio.run(main())
```

11. Build Order — Step by Step

Follow this exact order. Each step builds on the previous. Do NOT skip ahead — later modules depend on earlier ones being verified.

Step	What to Build	Files	Done When...
1	Project skeleton + config.py	All folders, <code>__init__.py</code> , <code>requirements.txt</code> , <code>config.py</code>	<code>pip install -r requirements.txt</code> succeeds with no errors
2	Database schema	<code>data/db.py</code>	<code>python -c 'import asyncio; from data.db import Database; asyncio.run(Database().init())'</code> creates <code>supervision.db</code> with correct tables
3	Camera capture	<code>vision/capture.py</code>	Running <code>capture.py</code> standalone opens webcam window at 15fps
4	MediaPipe module	<code>vision/mediapipe_module.py</code>	Running standalone shows face landmarks + prints <code>FaceResult/PoseResult</code> to console every second
5	YOLO module	<code>vision/yolo_module.py</code>	Running standalone detects a phone held in front of camera, prints confidence > 0.65
6	Optical flow	<code>vision/optical_flow.py</code>	Running standalone prints <code>writing_score</code> > 30 when hand moves on desk, < 10 when still
7	Feature aggregator	<code>vision/feature_aggregator.py</code>	Wires steps 4-6 together. Prints clean <code>FeatureVector</code> dict every second
8	Pomodoro state machine	<code>logic/state_machine.py</code>	Standalone test: machine transitions through all 6 states correctly in a mock 30-second run (use short durations in test)
9	Rules engine	<code>logic/rules_engine.py</code>	Feed mock <code>FeatureVectors</code> (<code>phone_detected=True</code> , etc.) and confirm correct <code>Events</code> emitted with correct priority and cooldown respected
10	Script bank + TTS	<code>output/script_bank.py</code> , <code>output/tts.py</code>	<code>script_bank.get('phone_detected_high', name='■■■')</code> returns Chinese string. <code>tts.speak()</code> plays audio
11	Session manager	<code>logic/session_manager.py</code>	Run a full 5-minute mock session (reduce <code>FOCUS_DURATION</code> in config). Bot speaks, events log to DB
12	FastAPI server	<code>api/server.py</code> , <code>api/schemas.py</code>	<code>curl http://127.0.0.1:8000/session/current</code> returns valid JSON
13	React dashboard	<code>dashboard/</code> (all files)	Dashboard shows live status, opens in browser at <code>localhost:5173</code> , <code>StatusCard</code> updates in real time
14	End-to-end test	<code>main.py</code>	Run <code>python main.py</code> . Sit in front of webcam for 2 minutes. Hold phone in frame. Confirm: voice alert fires, event appears in dashboard, DB has event row

12. Testing Checkpoints

These are the specific tests to run after each module is built. Write these as pytest test files in a tests/ folder.

12.1 Unit Tests

```
tests/
■■■ test_feature_aggregator.py
■ → Test: phone_detected=True in YoloResult → FeatureVector.phone_detected=True
■ → Test: EAR < 0.20 for 3 consecutive frames → ear_avg < 0.20 in vector
■
■■■ test_rules_engine.py
■ → Test: phone_detected=True for 6s → PHONE_DETECTED HIGH event emitted
■ → Test: phone_detected=True for 3s → no event (below PHONE_ALERT_SECONDS=5)
■ → Test: same event fired → blocked by cooldown (180s)
■ → Test: session events > MAX_ALERTS → only HIGH priority passes
■ → Test: no face for 70s → OUT_OF_SEAT event (threshold=60s)
■
■■■ test_state_machine.py
■ → Test: manual_start() → state changes IDLE → PREPARE
■ → Test: tick() for PREPARE_DURATION seconds → state changes to FOCUS
■ → Test: tick() for FOCUS_DURATION seconds → state changes to BREAK
■ → Test: force_break() from FOCUS → state changes to BREAK immediately
■
■■■ test_script_bank.py
■ → Test: get("phone_detected_high", name="■■■") returns string containing "■■■"
■ → Test: repeated calls return different scripts (randomness)
■ → Test: missing key raises KeyError with helpful message
■
■■■ test_db.py
  → Test: init() creates all 4 tables
  → Test: start_session() returns integer session_id
  → Test: log_event() inserts row retrievable by get_events()
```

12.2 Manual Integration Test Script

Run this manual sequence to verify the full pipeline before shipping:

```
MANUAL TEST CHECKLIST
=====
■ 1. Start system: python main.py --debug --name ■■ --age 10
■ 2. Dashboard loads at http://localhost:5173 – shows "Standby" state
■ 3. Click Start in dashboard – bot says session start script, state = PREPARE
■ 4. Sit in front of camera – state transitions to FOCUS within 30s
■ 5. Hold phone in front of camera for 6+ seconds – voice alert fires
■ 6. Check dashboard Event Timeline – phone event appears
■ 7. Look away from camera for 35+ seconds – distraction alert fires (medium)
■ 8. Cover camera with hand for 65+ seconds – out-of-seat alert fires (high)
■ 9. Wait for FOCUS_DURATION to expire (set to 2 min in test config)
■ 10. Bot says break script – state = BREAK
■ 11. Wait for SHORT_BREAK to expire – state = RESUME_CHECK
■ 12. Return to desk – state returns to FOCUS
■ 13. Click Stop – bot says session complete, state = COMPLETE
■ 14. Dashboard shows session summary with correct event counts
■ 15. Check data/supervision.db – verify Session, Event, and FocusSnapshot rows
```

12.3 False Positive Acceptance Criteria

Before declaring the software ready, run a 25-minute real study session and measure the false positive rate.

Acceptance criteria:

- Total voice alerts per session: fewer than 5 (not counting positive GOOD_FOCUS alerts)
- Alerts while genuinely working (reading, thinking, pausing): 0 per session
- Phone detection false positive rate: 0% (only fire if a phone is actually visible)
- Out-of-seat false positive from leaning back: 0 (slouch != absent)

13. Known Issues & Workarounds

Issue	Cause	Workaround / Fix
edge-tts fails silently	No internet connection	Always wrap in try/except. Fall back to pyttsx3. Pre-cache all template scripts at startup.
MediaPipe slow on CPU-only laptop	Full Face Mesh is heavy at 15fps	Use mediapipe.solutions.face_mesh with max_num_faces=1 and min_detection_confidence=0.5. Drop to 10fps if CPU > 80%.
YOLO model not found on first run	ultralytics downloads yolov8n.pt on first use, requires internet	Pre-download: run <code>python -c 'from ultralytics import YOLO; YOLO("yolov8n.pt")'</code> once during setup.
Optical flow wrong when child moves head	Global motion from head movement pollutes desk ROI flow	Subtract median flow of entire frame before summing ROI flow. This removes global camera/body motion.
FastAPI and CV loop conflict on Windows	asyncio and OpenCV conflict on Windows event loop	Use <code>asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())</code> at top of main.py on Windows.
EAR unreliable with glasses	Thick frames occlude eyelid landmarks	If face mesh confidence < 0.7 for EAR landmarks specifically, skip EAR check. Do not fire fatigue alert on low-confidence EAR readings.
TTS audio overlap	Alert fires while previous audio is still playing	Use <code>pygame.mixer.music.get_busy()</code> to check. If busy, skip new alert or queue it. Never overlap.

14. Quick Reference — Run Commands

```
# First-time setup
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python -c "from ultralytics import YOLO; YOLO('yolov8n.pt')" # Pre-download YOLO

# Start full system (CV + API + Dashboard backend)
python main.py --name ■■ --age 10 --debug

# Start API only (no camera, for dashboard testing)
python main.py --api-only

# Start React dashboard (separate terminal)
cd dashboard && npm run dev
# Open: http://localhost:5173

# Run tests
pytest tests/ -v

# Check DB contents
sqlite3 data/supervision.db ".tables"
sqlite3 data/supervision.db "SELECT * FROM events ORDER BY timestamp DESC LIMIT 10;"

# Clear all data (reset)
rm data/supervision.db && python -c "import asyncio; from data.db import Database; asyncio.run(Database().init())"
```

Summary: What This System Does When Running

1. CameraCapture reads frames at 15fps → thread-safe frame queue
2. MediaPipeModule processes each frame → FaceResult + PoseResult
3. YoloModule processes every 3rd frame → YoloResult (phone/person/book)
4. OpticalFlowModule processes each frame in desk ROI → writing_score
5. FeatureAggregator combines all into FeatureVector (rolling 5s window)
6. Every second:
 - a. PomodoroMachine.tick() → checks timer, fires state transitions
 - b. RulesEngine.evaluate(fv, state) → returns list of Events
 - c. For each Event: ScriptBank.get(key) → TTSEngine.speak(text)
 - d. Database.log_event() + Database.log_snapshot()
 - e. FastAPI SSE stream publishes latest FeatureVector
7. React dashboard polls /session/current every 5s + listens to SSE stream

End of Software Build Plan — Ready for Implementation